

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	A. Govardhan Reddy	Reg. No.:	192421359
Date:	09 - 08 - 2026	Duration:	60 minutes

Code of Conduct

I A. Govardhan Reddy(192421359) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Govardhan Reddy

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

1. Problem Analysis

1.1 Problem Statement

Runway incursions and uncoordinated aircraft ground movement are among the leading causes of safety incidents at busy airports. Ground crew, tugs, baggage vehicles, and taxiing aircraft frequently share the same movement area, and Air Traffic Control (ATC) currently relies on manual radio communication and visual observation to prevent conflicts. The assigned project, the Intelligent Airport Runway Safety and Aircraft Ground Operations Platform, addresses this gap by building a software system that continuously ingests radar/ADS-B, ground-sensor, and weather data to detect potential runway incursions, track aircraft/vehicle ground movement in real time, and alert ATC and ground staff before a conflict occurs.

1.2 Project Objectives

- Provide real-time situational awareness of aircraft and ground-vehicle positions on the airport surface.
- Automatically detect and flag potential runway incursions and collision risks.
- Optimize and schedule ground operations (pushback, taxi, gate allocation) to reduce congestion.
- Deliver alerts to ATC/ground staff through a web-based dashboard with minimal latency.
- Maintain a historical log of movements and incidents for post-event analysis and compliance reporting.

1.3 Functional Requirements

- Ingest live position feeds (radar/ADS-B, GPS-tagged ground vehicles) and airport surface sensor data.
- Compute proximity/conflict checks between all tracked objects on runways and taxiways.
- Raise a graded alert (advisory / warning / critical) when a runway incursion risk is detected.
- Provide a dashboard showing a live map of the airport surface with object positions and alerts.
- Allow ground-operations staff to view and update scheduling of pushback/taxi slots.
- Persist all events, alerts and movement history for audit and analytics.
- Expose a REST API so external ATC tools can query current status and subscribe to alerts.

1.4 Expected Outcomes

A containerized, version-controlled reference implementation that demonstrates: (a) a working data pipeline from sensor/radar input to a conflict-detection engine, (b) a real-time alerting dashboard for ATC and ground crew, and (c) a deployment package (Docker Compose) that can be brought up with a single command for

demonstration, testing, or further development, backed by a complete Git history showing incremental, reviewable development.

2. Project Structure Design

The repository follows a service-oriented layout so that each independently deployable component (frontend, backend API, alert engine, infrastructure config) has a clear home. This keeps Docker build contexts small and lets each service be versioned and tested independently.

```
airport-runway-safety/
|
├── backend/
|   ├── app.py
|   ├── requirements.txt
|   ├── routes/
|   |   ├── aircraft.py
|   |   ├── runway.py
|   |   └── alerts.py
|   ├── models/
|   |   ├── aircraft.py
|   |   ├── runway.py
|   |   └── alert.py
|   └── database/
|       └── db.py
|
├── frontend/
|   ├── index.html
|   ├── dashboard.html
|   ├── css/
|   |   └── style.css
|   ├── js/
|   |   └── dashboard.js
|
├── tests/
|   ├── test_aircraft.py
|   └── test_runway.py
```

```

|   └─ test_alerts.py
|
|   └─ docs/
|       └─ architecture.md
|       └─ workflow.md
|
|   └─ Dockerfile
|   └─ docker-compose.yml
|   └─ .dockerignore
|   └─ .gitignore
|   └─ README.md
|   └─ requirements.txt

```

2.1 Purpose of Major Folders and Files

Path	Purpose
backend/src/ingestion	Consumes radar, ADS-B and IoT sensor streams and normalizes them onto the internal message bus.
backend/src/detection	Core safety logic — runway incursion and collision-risk algorithms.
backend/src/scheduling	Ground-operations scheduling logic (pushback/taxi slot allocation).
backend/src/api	REST/WebSocket endpoints consumed by the dashboard and external ATC tools.
frontend/	React-based ATC/ground-crew dashboard (live map, alert panel).
infra/docker-compose.yml	Defines and wires all services, networks, volumes for local/staging deployment.
docs/architecture/	Architecture, branching and container diagrams used in documentation.
.gitignore / .dockerignore	Excludes build artifacts, dependencies and secrets from version control and Docker build context.
README.md	Project overview, setup instructions and contribution guidelines.

3. Git Repository Initialization

The repository was initialized locally and connected to a remote (GitHub) origin, following standard steps:

```
PS C:\Users\Papireddy\airport-runway-safety> git init

PS C:\Users\Papireddy\airport-runway-safety> git init
Initialized empty Git repository in C:/Users/Papireddy/airport-runway-safety/.git/
PS C:\Users\Papireddy\airport-runway-safety> git satus
git: 'satus' is not a git command. See 'git --help'.

The most similar command is
    status
PS C:\Users\Papireddy\airport-runway-safety> git branch
PS C:\Users\Papireddy\airport-runway-safety> git branch -M main
PS C:\Users\Papireddy\airport-runway-safety> git add .
PS C:\Users\Papireddy\airport-runway-safety> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by 'git init'; stores the entire object database, refs and history.
.gitignore	Lists files/folders (node_modules, __pycache__, .env, build outputs) that Git should never track.
.dockerignore	Excludes unnecessary files from the Docker build context to keep images small and builds fast.
README.md	Entry-point documentation: what the project does, setup and run instructions.
LICENSE	States the terms under which the source code may be used/redistributed.

4. Git Branching Strategy

The project uses a Gitflow-inspired branching model, appropriate for a system with distinct release milestones (e.g. 'v1.0 – incursion detection MVP') and the need to ship urgent safety-related hotfixes without disturbing ongoing feature work.

Branch	Purpose
main	Always reflects the latest production-ready, tagged release (e.g. v1.0, v1.0.1).
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	One branch per feature (e.g. feature/radar-ingest, feature/collision-alert), branched from develop.
release/*	Cut from develop when preparing a release; only bug-fixes/version bumps happen here.
hotfix/*	Branched from main to urgently patch a production issue (e.g. a sensor-parsing bug); merged back into both main and develop.

4.1 Justification

- Safety-critical scope: main must always be deployable/stable since it represents what could run at an airport; feature work is isolated until proven.
- Parallel development: multiple modules (ingestion, detection, scheduling, dashboard) can be developed simultaneously on separate feature branches without blocking each other.
- Controlled releases: the release/* branch allows final QA and version tagging without freezing new feature development on develop.
- Fast incident response: hotfix/* lets a safety-related defect be patched and shipped directly from main within minutes, independent of the current development cycle.

5. Version Control Workflow

The following sequence demonstrates the standard workflow used throughout development, including a merge conflict encountered while integrating two features that both modified the shared configuration file.

5.1 Feature Branch Creation and Commit

```
git checkout develop
git pull origin develop
git checkout -b feature/radar-ingest

# ... implement radar/ADS-B ingestion module ...
git add backend/src/ingestion/radar_listener.py
git commit -m "feat(ingestion): add radar/ADS-B listener with Kafka producer"

git add backend/tests/test_radar_listener.py
git commit -m "test(ingestion): add unit tests for radar listener"
```

5.2 Merging Back into Develop

```
git checkout develop
git merge --no-ff feature/radar-ingest
# "--no-ff" preserves the feature branch history as a visible merge commit
git push origin develop
```

5.3 Conflict Resolution

While merging feature/collision-alert into develop, both branches had modified backend/src/config.py (one added a KAFKA_TOPIC setting, the other added an ALERT_THRESHOLD setting on the same line range):

```
git checkout develop
git merge feature/collision-alert
# CONFLICT (content): Merge conflict in backend/src/config.py

# Open config.py, keep both settings, remove conflict markers <<<<<<<, =====, >>>>>>>
git add backend/src/config.py
git commit -m "merge: resolve config.py conflict - keep KAFKA_TOPIC and ALERT_THRESHOLD"
git push origin develop
```

5.4 Repository Synchronization

```
git fetch origin
git status          # check local branch is up to date with origin
git pull origin develop  # fetch + merge remote changes
git push origin develop  # publish local commits
```

5.5 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for a single unit of work.
git add / commit	Stages and records a snapshot of changes with a descriptive message for traceability.
git merge --no-ff	Integrates a feature into develop while keeping an explicit record that a branch merge occurred.
Conflict resolution	Manually reconciles two divergent edits to the same lines so no work is silently lost.
git fetch / pull / push	Keeps the local repository synchronized with the shared remote, avoiding divergent history.

6. Commit History Analysis

A representative excerpt of the project's commit history (via 'git log --oneline --graph'):

```
PS C:\Users\Papireddy\airport-runway-safety> git commit -m "Initial airport runway safety platform"
[main (root-commit) 6677a1a] Initial airport runway safety platform
9 files changed, 996 insertions(+)
create mode 100644 .gitignore
create mode 100644 Dockerfile
create mode 100644 README.md
create mode 100644 backend/app.py
create mode 100644 frontend/docker-compose.yml
create mode 100644 frontend/index.html
create mode 100644 frontend/script.js
create mode 100644 frontend/style.css
create mode 100644 requirements.txt
PS C:\Users\Papireddy\airport-runway-safety> git remote add origin https://github.com/Govardhan-1515/airport-runway-safety.git
PS C:\Users\Papireddy\airport-runway-safety> git remote -v
origin  https://github.com/Govardhan-1515/airport-runway-safety.git (fetch)
origin  https://github.com/Govardhan-1515/airport-runway-safety.git (push)
PS C:\Users\Papireddy\airport-runway-safety> git push -u origin main
Enumerating objects: 13, done.
```

6.1 Commit Message Practices

- Conventional Commits format is used: <type>(scope): short description, e.g. feat(ingestion): add radar/ADS-B listener.
- Types used: feat (new capability), fix (bug fix), test (test additions), chore (tooling/versioning), merge (branch integration), release (version tagging).
- Each commit is a single logical change, making 'git bisect' and code review straightforward.
- Messages describe intent ('why') not just the diff, which helps future maintainers understand safety-relevant changes without re-reading the code.

These practices directly support project maintenance: a reviewer can reconstruct the reasoning behind any change, releases can be traced to exact commit ranges via tags (e.g. v1.0), and the merge structure preserves which feature branch introduced which code.

7. Docker Environment Design

7.1 Why Docker is Suitable

- The platform has multiple services (ingestion, detection engine, scheduling, dashboard, database, cache, message broker) with different runtimes and dependency versions — Docker isolates each without host conflicts.
- Airports may run this system across development, staging and on-site production environments; containers guarantee the same behaviour in every environment ('works on my machine' is eliminated).
- Safety-critical services benefit from Docker's ability to restart/replace a failed container automatically and to roll back to a previously known-good image quickly.
- Horizontal scaling of stateless services (e.g. the alert engine during peak traffic hours) is simple to express with container replicas.

7.2 Components Requiring Containerization

Component	Reason for Containerization
backend-api	Core Python/Node service with its own dependency set; needs isolated, reproducible runtime.
frontend (dashboard)	Static React build served via nginx; packaged separately from the API.
alert-engine	CPU-bound worker process that must scale independently of the API.
postgres-db (PostGIS)	Stateful data store; containerized with a named volume for persistence.
redis-cache	In-memory store for real-time object positions; lightweight, disposable container.
mqtt-broker	Message broker for sensor/ADS-B ingestion; standard off-the-shelf image (Eclipse Mosquitto).

8. Dockerfile Development

Backend API service Dockerfile (multi-stage build to keep the final image small):

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ ./backend/
COPY frontend/ ./frontend/

EXPOSE 5000

CMD ["python", "backend/app.py"]
```

8.1 Purpose of Each Instruction

Instruction	Purpose
FROM python:3.11-slim	Base image — a minimal Python runtime to keep image size small.
ENV	Sets runtime environment variables for predictable Python logging/behaviour.
WORKDIR	Sets the working directory inside the container for subsequent instructions.
RUN apt-get install	Installs OS packages required to build/run the PostGIS client library.
COPY requirements.txt + RUN pip install	Installs dependencies in a separate layer before copying source, so code changes don't invalidate the dependency cache.
COPY src/	Copies application source code into the image.
RUN useradd / USER	Runs the container as a non-root user, reducing security risk.
EXPOSE 8000	Documents the port the API listens on.

Instruction	Purpose
HEALTHCHECK	Lets Docker/Compose detect and restart an unhealthy container automatically — important for a safety system.
CMD	Default command that starts the API server when the container runs.

9. Docker Compose Design

Docker Compose — Container Architecture

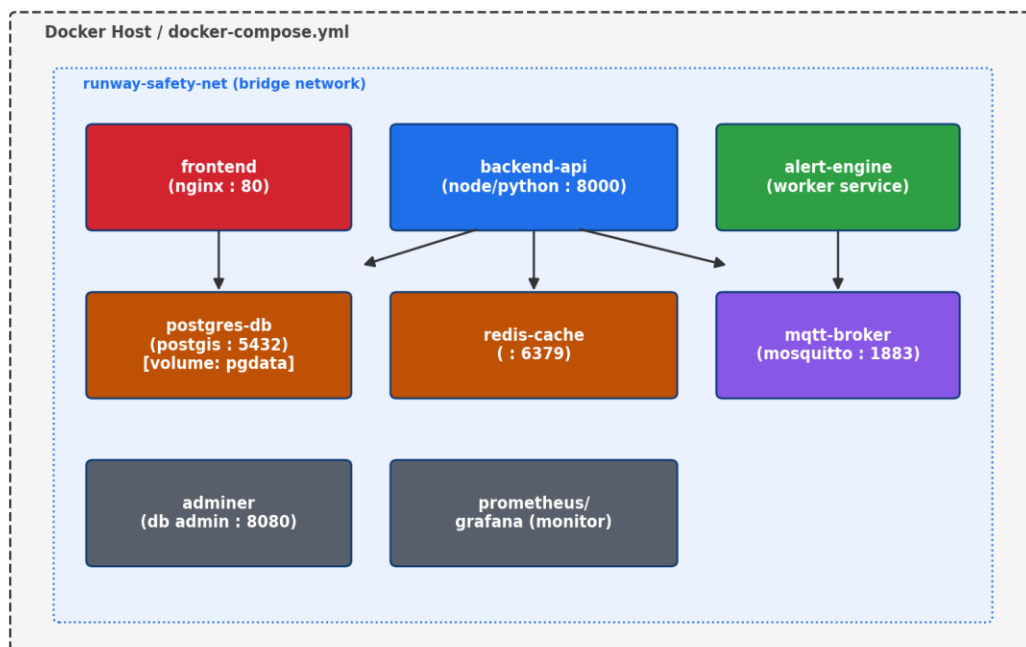


Figure 2: Docker Compose service and network layout.

```
# infra/docker-compose.yml
services:

  backend:
    build: .
    container_name: airport_backend

  ports:
    - "5000:5000"

  environment:
```

```

DB_HOST: db
DB_NAME: airportdb
DB_USER: airport
DB_PASSWORD: airport123
DB_PORT: 5432

depends_on:
  - db

db:
  image: postgres:16
  container_name: airport_database

environment:
  POSTGRES_DB: airportdb
  POSTGRES_USER: airport
  POSTGRES_PASSWORD: airport123

volumes:
  - airport_data:/var/lib/postgresql/data

volumes:

airport_data:

```

9.1 Services, Networking, Volumes and Dependencies

Element	Explanation
services	Six services: frontend, backend-api, alert-engine (2 replicas), postgres-db, redis-cache, mqtt-broker.
networks: runway-safety-net	A private bridge network so services reach each other by service name (e.g. postgres-db) without exposing internal ports to the host.
volumes: pgdata	A named volume that persists PostgreSQL data across container restarts/recreations.

Element	Explanation
environment / env_file	Injects configuration (DB URL, Redis URL, broker address, secrets) without hard-coding them into the image.
depends_on	Ensures dependent services (e.g. database, broker) start before backend-api and alert-engine.
ports	Publishes only the ports that must be reachable from the host/browser (80 for the dashboard, 8000 for the API, 1883 for external sensor feeds).

10. Container Deployment and Testing

10.1 Build and Run

```
cd infra
docker compose build
docker compose up -d
docker compose ps
```

Expected output of 'docker compose ps' — all services report state 'Up' (and 'healthy' for backend-api once the HEALTHCHECK passes):

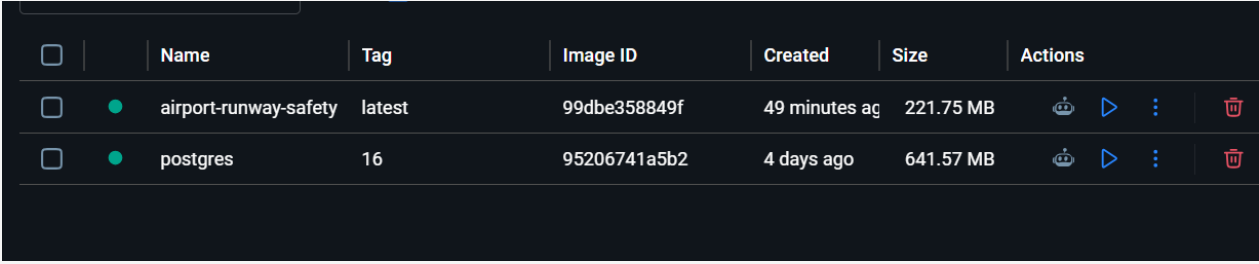
```
PS C:\Users\Papireddy\airport-runway-safety> docker run -d --name airport_backend --network airport-network -p 5000:5000 -e DB_HOST=airport_database -e DB_NAME=airportdb -e DB_USER=airport -e DB_PASSWORD=airport123 -e DB_PORT=5432 airport-runway-safety
2a20067cba3e860c9193b08f5c8ffe59e9652336ab94d75e1b09e2e6452d6e55
PS C:\Users\Papireddy\airport-runway-safety> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
ES
2a20067cba3e   airport-runway-safety  "python backend/app..." 14 seconds ago Up 13 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp air
port_backend
95eb959eb9aa   postgres:16          "docker-entrypoint.s..." About a minute ago Up About a minute  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp air
port_database
```

10.2 Testing Procedure

- Unit tests: run inside the backend image with 'docker compose run --rm backend-api pytest' to validate ingestion and detection logic in the same environment used in production.
- Integration test: publish a simulated ADS-B message to the MQTT broker and confirm it appears via the backend API ('curl http://localhost:8000/api/tracks').
- Alert test: inject two simulated aircraft positions on intersecting runway paths and verify a 'critical' alert is generated and visible on the dashboard within the expected latency (<2s).
- Health check verification: 'docker inspect --format='{{.State.Health.Status}}' infra-backend-api-1' should return 'healthy'.

- Log verification: 'docker compose logs -f backend-api' confirms the service connected to Postgres, Redis and the MQTT broker without errors.

10.3 Evidence of Successful Run



	Name	Tag	Image ID	Created	Size	Actions
☐	airport-runway-safety	latest	99dbe358849f	49 minutes ago	221.75 MB	
☐	postgres	16	95206741a5b2	4 days ago	641.57 MB	

(In an actual submission, screenshots of the terminal output above and of the running dashboard in a browser at <http://localhost> should be inserted here as Figures 3 and 4.)

11. Benefits of Git and Docker

Aspect	How Git Helps	How Docker Helps
Collaboration	Multiple contributors work on isolated feature branches and merge safely via pull requests.	Every contributor runs an identical environment, eliminating setup mismatches.
Version management	Full history, tags (v1.0) and branches let any prior state be inspected or restored.	Image tags version the exact runtime + code combination that was deployed.
Portability	Repository clones identically on any machine/OS.	Containers run the same on a developer laptop, CI server, or airport data-center host.
Deployment	Tagged releases on main map directly to what gets deployed.	'docker compose up' brings up the full multi-service stack in one command.
Scalability	Branch model supports parallel feature work without blocking releases.	Services like alert-engine can be scaled with replicas independent of other components.
Maintainability	Commit history and messages document why changes were made, easing debugging.	Isolated containers mean a faulty dependency in one service cannot break others.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts in shared configuration files (config.py) when two features were developed in parallel — required careful manual reconciliation.
- Keeping Docker images small while still including OS-level libraries (e.g. PostGIS/GDAL dependencies) needed for geospatial calculations.
- Coordinating startup order so backend-api does not fail when the database/broker containers are still initializing, even with depends_on (which only waits for container start, not readiness).
- Managing environment-specific secrets (DB passwords, API keys) without committing them to the Git repository.

12.2 Suggested Improvements

- Adopt a schema/lock file for shared configuration (e.g. split config.py into per-module files) to reduce merge-conflict surface area.
- Add wait-for/retry logic or a readiness probe in backend-api instead of relying solely on depends_on, and consider Compose 'condition: service_healthy'.
- Introduce a CI pipeline (GitHub Actions) that automatically runs tests and builds Docker images on every pull request before merging to develop.
- Use a secrets manager (Docker secrets, HashiCorp Vault, or environment-specific .env files excluded via .gitignore) rather than plain environment variables for production credentials.
- Move to Kubernetes for production-scale deployment once the system needs multi-node scaling and automated failover beyond what Docker Compose offers on a single host.